

JEGYZŐKÖNYV

Modern adatbázis rendszerek MSc

LINQ Programozás

Készítette: **Kiss Konrád Soma**

Neptunkód: **CNY8MP**

Dátum: **2026. május. 04.**

Tartalomjegyzék

1. A LINQ Programozás témaköre	3
1.1. A témakör elmélete röviden	3
1.2. Az én adatbázis mintám	4
2. Az előadás leírása	6
2.1. Az előadás tervezésének lépései	6
3. A gyakorlati feladat leírása	7
3.1. A feladat tervezésének lépései	7
3.2. A futtatás eredménye	7
4. Mellékletek	8

1. A LINQ Programozás témaköre

1.1. A témakör elmélete röviden

A LINQ (Language Integrated Query) egy olyan technológia, amely lehetővé teszi adatok strukturált lekérdezését közvetlenül a C# kódban. A LINQ célja, hogy egységes szintaxist biztosítson az adatforrások lekérdezéséhez, függetlenül attól, hogy XML, relációs adatbázis vagy memóriában tárolt kollekcióban találhatók az adatok.

A LINQ-XML modul speciálisan az XML dokumentumok feldolgozásához lett kifejlesztve. Az XDocument és XElement osztályok segítségével könnyedén navigálhatunk az XML fastruktúrában, lekérdezhünk adatokat, szűrhetünk és transzformálhatunk tartalmakat.

Alapvető előnyei:

- Erős típusosság: a fordítási időben ellenőrzik a lekérdezéseket
- IntelliSense támogatás az IDE-ben
- Deferred execution: a lekérdezések csak igény szerint hajtódnak végre
- Hatékony szűrés, csoportosítás és aggregáció

Fontos LINQ fogalmak röviden:

- **Lekérdezés szintaxis vs. metódus lánc:** A LINQ két, egymással ekvivalens kifejezési formát támogat. A metódus lánc (pl. `.Where(...).Select(...)`) közvetlenül a kiterjesztő metódusokra épül, míg a query szintaxis (`from ... where ... select ...`) egy olvashatóbb „szintaktikus cukor”.
- **Deferred execution:** Sok LINQ lekérdezés csak akkor fut le ténylegesen, amikor bejárjuk (pl. `foreach`) vagy materializáljuk (`ToList()`, `ToArray()`). Ez hasznos teljesítményben, de hibákhoz is vezethet, ha a forrás közben megváltozik.
- **Null-biztonság:** XML feldolgozásnál gyakori, hogy egy elem/attribútum hiányzik. Emiatt célszerű a `?.` operátort használni és alapértékekkel védekezni.

LINQ-XML esetén tipikus csapdák:

- XML elemnevekben szerepelhet kötőjel (pl. `utca-hazszam`, `netto-ar`, `datum-ido`) és ékezet (pl. `fuvarozó`). Ezeket a LINQ to XML stringként ugyanígy kell megadni.
- A `.Descendants(...)` az egész fa alatt keres, míg az `.Elements(...)` csak közvetlen gyerekeket ad vissza. A feladatnál a gyökér (`<uzeletek>`) közvetlen gyerekeinek kezelése általában áttekinthetőbb.

Példa (query szintaxis):

```
// Mely kereskedések vannak Budapesten?  
var budapestiek =  
    from k in root.Elements("kereskedes")  
    let cim = k.Element("cim")  
    where (string?)cim?.Element("varos") == "Budapest"  
    select (string?)k.Attribute("k_id");
```

Példa (metódus lánc):

```
var raktarakSzama = root.Elements("raktar").Count();
```

1.2. Az én adatbázis mintám

A gyakorlatban egy kereskedelmi rendszer adatbázisát használjuk, amely a következő entitásokból áll:

Kereskedések (kereskedes): Az üzleteink, amelyeknek város, utca, házszám és irányítószáma van. Azonosítóval (k_id) rendelkeznek.

Raktárak (raktar): Az áru tárolásának helyei, szintén cím-adatokkal. Azonosítóval (r_id).

Áru (aru): A kereskedhető termékek neve, nettó ára és áfa-tartalma.

Alkalmazottak (alkalmazott): A kereskedésekben dolgozó személyzet.

Beszállítók (beszallito): Raktárak és kereskedések közötti kapcsolatok.

Leltár (leltar): Megadja, hogy melyik raktárban (r_fid) melyik áru (a_fid) hány darabban (darabszam) érhető el.

Rendelés (rendeles): Egy rendelés azonosítóval (re_id) rendelkezik, és összekapcsolja a fuvarozót (f_fid), a vásárlót (v_fid) és a kereskedést (k_fid). Tartalmazza a teljesülés állapotát, dátum-időt és a teljes árat.

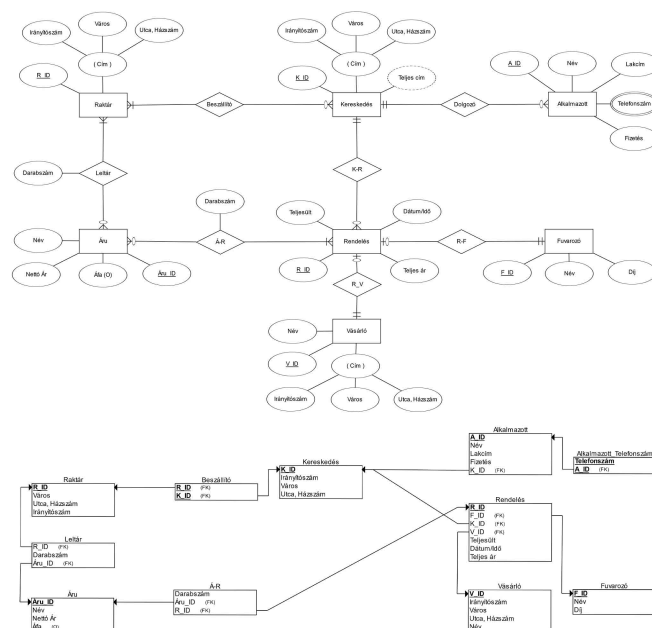
Áru–Rendelés kapcsolat (a-r): A rendelés tételsora (melyik áruból mennyi darab került a rendelésbe). Kapcsoló attribútumok: a_fid és re_fid.

Vásárló (vasarlo): A vásárlók neve és címe, azonosítóval (v_id).

Fuvarozó (fuvarozó): A szállítást végző cégek neve és díja, azonosítóval (f_id).

Az entitások közötti kapcsolatok lényege:

- A kereskedes elemen belül a dolgozo a_fid="..." elemek **hivatkoznak** az alkalmazott a_id="..." elemekre.
- A beszallito összeköti a raktar és kereskedes rekordokat (r_fid ↔ r_id, k_fid ↔ k_id).
- A leltar összeköti az aru és raktar rekordokat (a_fid ↔ a_id, r_fid ↔ r_id).
- A rendeles hivatkozik a vasarlo és fuvarozó elemekre, továbbá a kereskedes elemre.
- Az a-r adja meg a rendelés tételeit (árúk és darabszámok).



Az XML szerkezet:

```
<uzeletok>
  <kereskedes k_id="k1">
    <cim>
      <iranyitoszam>1234</iranyitoszam>
      <varos>Budapest</varos>
      <utca-hazszam>Váci utca 10.</utca-hazszam>
    </cim>
    <dolgozo a_fid="a1"/>
  </kereskedes>
  <raktar r_id="r1">
    <cim>
      <iranyitoszam>4321</iranyitoszam>
      <varos>Szeged</varos>
      <utca-hazszam>Raktár utca 3.</utca-hazszam>
    </cim>
  </raktar>
  <aru a_id="a1">
    <nev>Tej</nev>
    <netto-ar>200</netto-ar>
    <afa>27</afa>
  </aru>
  <leltar a_fid="a1" r_fid="r1">
    <darabszam>10</darabszam>
  </leltar>
  <vasarlo v_id="v1">...</vasarlo>
  <fuvarozó f_id="f1">...</fuvarozó>
  <rendeles re_id="re1" f_fid="f1" v_fid="v1" k_fid="k1">...</rendeles>
  <a-r a_fid="a1" re_fid="re1">...</a-r>
</uzeletok>
```

Megjegyzés: a fenti XML részlet csak szemléltető snippet; a teljes minta a mellekletek/dbschema.xml fájlban található.

2. Az előadás leírása

2.1. Az előadás tervezésének lépései

Az előadás célja a hallgatók megismertetése a LINQ-XML technológiával, amely lehetővé teszi XML dokumentumok hatékony feldolgozását C# kódban.

Az előadás három fő részből áll:

1. Elméleti alapok: A LINQ fundamentális koncepciói, az XDocument és XElement osztályok, valamint a lekérdezés szintaxis alapjainak ismertetése.

2. Praktikus alkalmazások: Valós példákon keresztül megmutatjuk, hogyan lehet XML fájlokat betölteni, adatokat szűrni, csoportosítani és transzformálni.

3. Önálló gyakorlatok: A hallgatók saját kódot írnak a tanult koncepciók alkalmazásához.

Az adatbázis modelljét a kereskedelmi rendszerre alapoztam, mivel könnyen érthető, valósághoz közeli, és megfelelően összetett a LINQ előnyeinek bemutatásához.

Az előadás felépítését az alábbi lépések szerint terveztem meg:

- Követelmények tisztázása:** milyen fejlesztőkörnyezetet használunk, hol található a minta XML, milyen lekérdezések a kötelezőek.
- Adatmodell áttekintése:** a minta XML fő elemeinek azonosítása (kereskedes, raktar, aru, stb.), azonosítók és hivatkozások értelmezése.
- Technikai alapok bemutatása:** XDocument.Load, Root, .Elements, .Descendants, valamint a string? konverziók és a ?. null-biztonság.
- Mintalekérdezések demonstrálása:** szűrés (Where), transzformáció (Select), csoportosítás (GroupBy), összekapcsolás hivatkozások alapján (dictionary/lookup).
- Hibakezelés és adatminőség:** hiányzó attribútumok/elemek kezelése, számmá alakítás (int, decimal) és alapértékek.
- Önálló feladat:** a hallgatók a minták alapján megoldják a feladatokat, majd az eredményt röviden interpretálják.

3. A gyakorlati feladat leírása

3.1. A feladat tervezésének lépései

1. **Fejlesztési környezet:** A hallgatók Visual Studio Code és .NET SDK 8.0-t használnak.

2. Alapvető feladatok:

1. Az XML dokumentum betöltése `XDocument.Load()` segítségével (pl. `etterem.xml`)
2. A gyökérelem (vendéglátás) kiválasztása és a teljes dokumentum kiírása
3. Egyszerű szűrés LINQ segítségével: az ötcsillagos éttermek listázása

3. Haladó feladatok:

1. Komplex lekérdezés több „JOIN” jellegű összekapcsolással (vendég ↔ rendelés ↔ étterem)
2. Aggregáció: az átlagos költség meghatározása a rendelések összegeiből
3. Módosítás: minden rendelés összegének megduplázása és mentése új XML fájlba (`etterem_modositott.xml`)
4. Törlés: a 3 csillagos éttermek eltávolítása és mentése új XML fájlba (`etterem_torolt.xml`)

3.2. A futtatás eredménye

Az alkalmazás futtatásakor (a mintaadatok alapján) az alábbi jellegű kimenet jelenik meg:

```
=== 0) A teljes dokumentum ===
<vendéglátas>...
```

```
=== 1) Az ötcsillagos éttermek ===
- Arany Kanál Étterem
- Panoráma Bistro
```

```
=== 2) Ki-mit rendelt, hol, mennyiért ===
- Vendég: Kovács János | Étterem: Arany Kanál Étterem | Étél: Gulyásleves | Összeg: 2490
- Vendég: Szabó Péter | Étterem: Panoráma Bistro | Étél: Rántott sajt | Összeg: 2790
```

```
=== 3) Az átlagos költség ===
- Átlag: 3125,50
```

```
=== 4) Mentés (módosított) ===
- Az új fájl neve: "etterem_modositott.xml"
```

```
=== 5) Mentés (törölt 3 csillagos éttermek nélkül) ===
- Az új fájl neve: "etterem_torolt.xml"
```

4. Mellékletek

A mellékletben található:

- `mellekletek/dbschema.xml`: a kereskedés minta (üzletek/raktárak/leltár/rendelések stb.)
- `mellekletek/dbschema.jpg`: az adatmodell ábrája (vizuális séma/diagram)
- `mellekletek/c_sharp_program/`: egy korábbi, XML LINQ-os C# mintaprojekt (a benne lévő `etterem.xml` alapú példa nem azonos ezzel a kereskedés mintával)

A `dbschema.xml` a `CNY8MP_XML_hazifeladat.xsd` sémára hivatkozik; a sémafájl megtalálható a projektben (pl. a `CNY8MP_0318/` mappában).